- Review III – Panel Review Overall Report (20 Marks)
  - 1. Implementation (5 Marks)
  - 2. Technical Accuracy (5 Marks)
  - 3. Results Obtained So Far (5 Marks)
  - 4. Presentation and Clarity (5 Marks)

Review III – Panel Review Overall Report (20 Marks)

This report maps the current state of the Adaptive Memory Management Capstone Project directly to the university's Review III Rubric criteria, providing comprehensive evidence for maximum score allocation.

---

1. Implementation (5 Marks)

Rubric Requirement: Demonstrates working modules representing approximately half of the approved scope; the evidence is executable and attributable to the team.

Project Status: Phase 1 Fully Completed (100% of Half-Scope Executable)
The project currently possesses a fully executable Python-based prototype encompassing the entire data-to-decision pipeline.

- **Executable Evidence:**
  - `python scripts/live_demo.py` executes an 8-step pipeline in 15 seconds, loading data, invoking the trained XGBoost model, calibrating probabilities, and simulating the OS memory controller.
  - `python -m streamlit run dashboard/app.py` launches a 10-page interactive visualization suite proving the modularity of the system.
- **Working Modules Completed:**
  1. **Data Pipeline:** Synthetic generator (Markov-chain based) yielding 50,000 events across 5 personas, cross-validated against 6 Kaggle datasets. Real-hardware data collection modules implemented via ADB (Android) and `ctypes` (Windows).
  2. **Prediction Engine:** XGBoost model trained on 19 temporal/contextual features achieving 67% Top-1 accuracy.
  3. **Confidence Calibrator:** Platt Scaling module successfully mapping raw logits to true probabilities (ECE reduced by 94%).
  4. **Online Learner:** Concept drift detector using KL-Divergence and PSI to trigger background sliding-window retraining.
  5. **Simulation Engine:** Real-time OS memory simulator benchmarking NoOpt, LRU, Static Preload, and the Proposed 6-Factor Policy.

---

2. Technical Accuracy (5 Marks)

Rubric Requirement: Uses technically correct methods, algorithms, parameters, and implementation practices consistent with the approved design.

Algorithmic Rigor & OS Alignment:

- **Bridging ML and OS Constraints:** The primary technical achievement is preventing the ML model from crashing the operating system. Instead of allowing XGBoost to blindly preload apps, predictions are mathematically gated.
- **Preload Gate & Platt Scaling:** By calibrating XGBoost probabilities via Logistic Regression (Platt Scaling), the system guarantees that a "60% confidence" score implies a strict 60% statistical likelihood. The OS 'Preload Gate' rejects any prediction below this calibrated threshold, preventing page faults and RAM thrashing.
- **The 6-Factor Cache Score:** Standard OS systems use deterministic algorithms like LRU (Least Recently Used). This project mathematically fuses deterministic OS heuristics with probabilistic ML foresight via the formula: $S = w_R R + w_F F + w_P P + w_I I + w_C C - w_M M$. The negative weight for Memory Pressure ($w_M = 0.30$) acts as a critical technical safeguard against OOM (Out Of Memory) kills.
- **Testing & Integrity:** The codebase maintains extreme technical accuracy, validated by 50 passing unit tests and strict adherence to SOLID object-oriented principles.

---

3. Results Obtained So Far (5 Marks)

Rubric Requirement: Presents interim metrics, tables, graphs, or outputs and provides technically sound interpretation and comparison.

Statistically Significant Performance Gains:

- **Simulation Benchmarks (120 Runs):** Across 120 simulated runs varying by persona and RAM constraint (4GB, 6GB, 8GB), the proposed policy averaged an **83.0% Cache Hit Rate** versus standard Android LRU at 78.2% (+4.8% global average).
- **Constrained Environment Supremacy:** The system excels under maximum pressure. On a 4GB RAM device under the 'Power User' persona, the proposed policy outperformed LRU by **+6.7%**, dramatically reducing cold-start latencies.
- **Ablation Study Integrity:** The project provides a component-by-component ablation study (viewable in the Dashboard). It proves that removing the Frequency factor degrades performance most severely, confirming the necessity of merging standard OS heuristics with the new ML prediction component.
- **Statistical Proof:** Paired t-tests run against the baseline models yielded p-values < 0.001 with Cohen's d effect sizes > 1.7, mathematically proving the improvements are systemic and not due to random chance.

---

4. Presentation and Clarity (5 Marks)

Rubric Requirement: Explains the work logically, justifies decisions, and responds accurately to panel questions.

Strategic Project Justifications:

- **Why Synthetic Data?** To test edge-case OS memory behavior (zero battery, thrashing), massive localized datasets are required. Synthetic generation (validated against published literature) allowed simulation of 180 days of extreme edge cases across varied personas, which is practically impossible to collect safely via manual ADB tethering. The 367 self-collected events serve as physical hardware validation, not the training corpus.
- **Why XGBoost over Deep Learning?** The system must execute during the milliseconds of an app switch. XGBoost achieves ~0.05ms inference times with a ~200KB footprint. Using LSTMs or deep transformers would consume 10-50MB of RAM, ironically causing the exact memory pressure the system was built to prevent.

Phase 2 Roadmap (Next Review Scope):

1. **App2Vec (Graph Embeddings):** Transitioning from categorical label encoding to Node2Vec temporal graph embeddings to map semantic app relationships.
2. **PrefixSpan Algorithm:** Implementing sequence mining to detect deep temporal patterns.
3. **C++ Native Integration:** Porting the Python inference engine to C++ (via TF Lite/ONNX) to map out theoretical hooks into the Android `ActivityManagerService` layer.
4. **Patent Filing:** Formalizing the intellectual property by completing Indian Patent Office (IPO) Forms 1, 2, and 5 based on the Confidence-Gated Preload architecture.